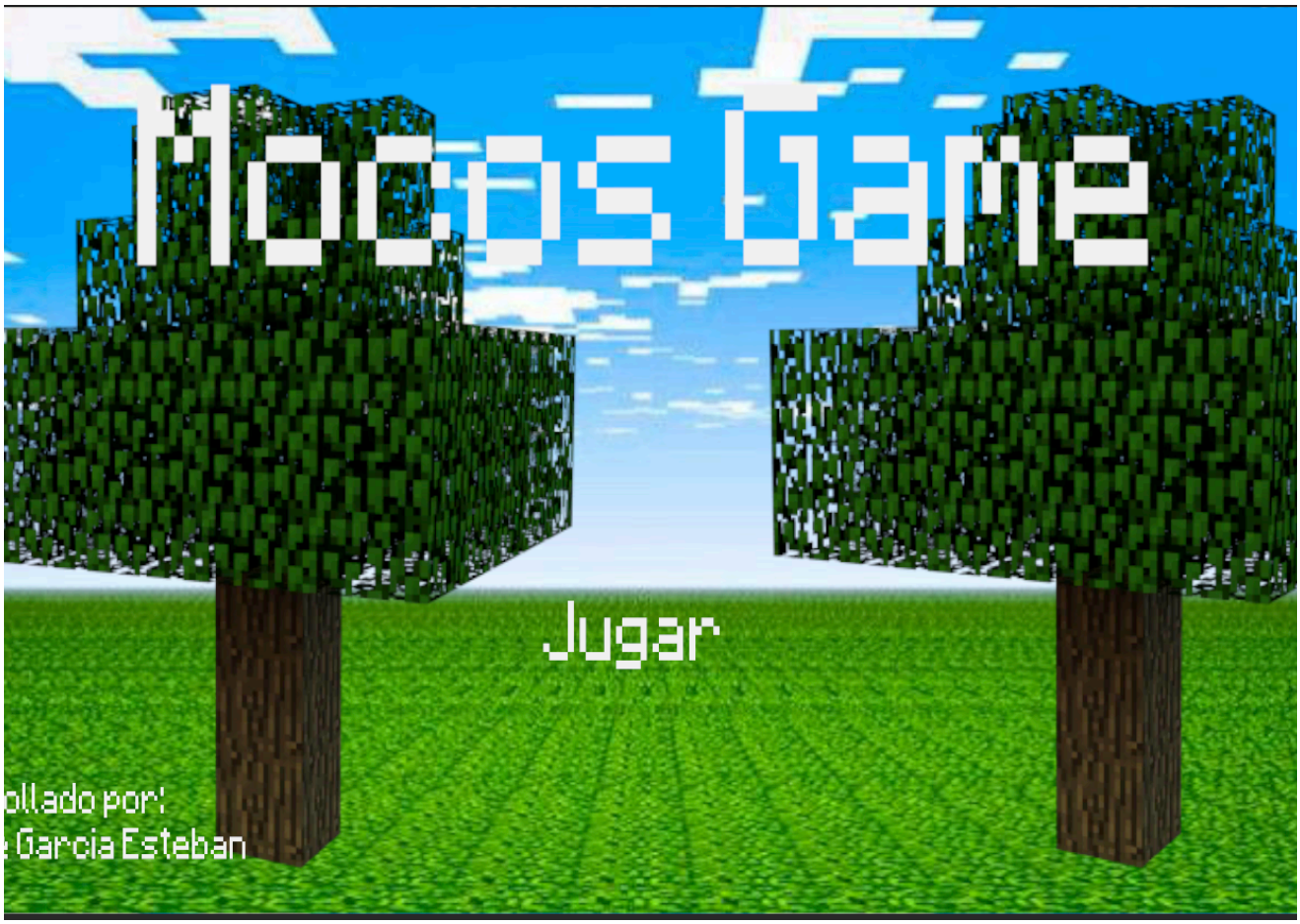
Tarea 12 PMDM

Creación de un juego en Unity

Emilio José García Esteban - 15 de abril de 2026



Introducción.

He creado un videojuego según me dio la idea mi hijo de 10 años, él es fanático de Minecraft y el juego en base esta ambientado en un personaje (Moco) que va recorriendo saltando las plataformas sumando puntos si coge picos y restando si coge dinamita.

El juego en sí no es gran cosa comparado a los que hay hoy en día en el mercado, pero he de decir que no me ha resultado fácil hacer la tarea debido a muchos problemas que he tenido en el tiempo que la he realizado. Este proyecto que entrego es el intento 6 o 7 sin contar con las prácticas que hice con el corre corre del mapache.

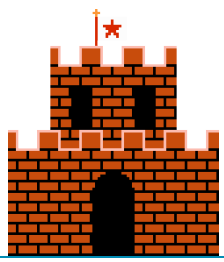
Casi todos los problemas que me he encontrado han sido debidos a la versión de unity, los pasos descritos en la guía que se proporciona no corresponde a la versión de unity actual, en mi caso la versión 6.4, están las opciones cambiadas de sitio o inclusive inexistentes. Por ejemplo a la hora de hacer la cámara de game over las “clear flags” ya no existen, hay que poner una cámara derivada de otra y tocar otras opciones para que aparezca de esa forma como “sombreado” encima de la pantalla principal.

Dicho esto paso a describir los pasos que he seguido para la realización de la tarea.

Preparación de los sprites.

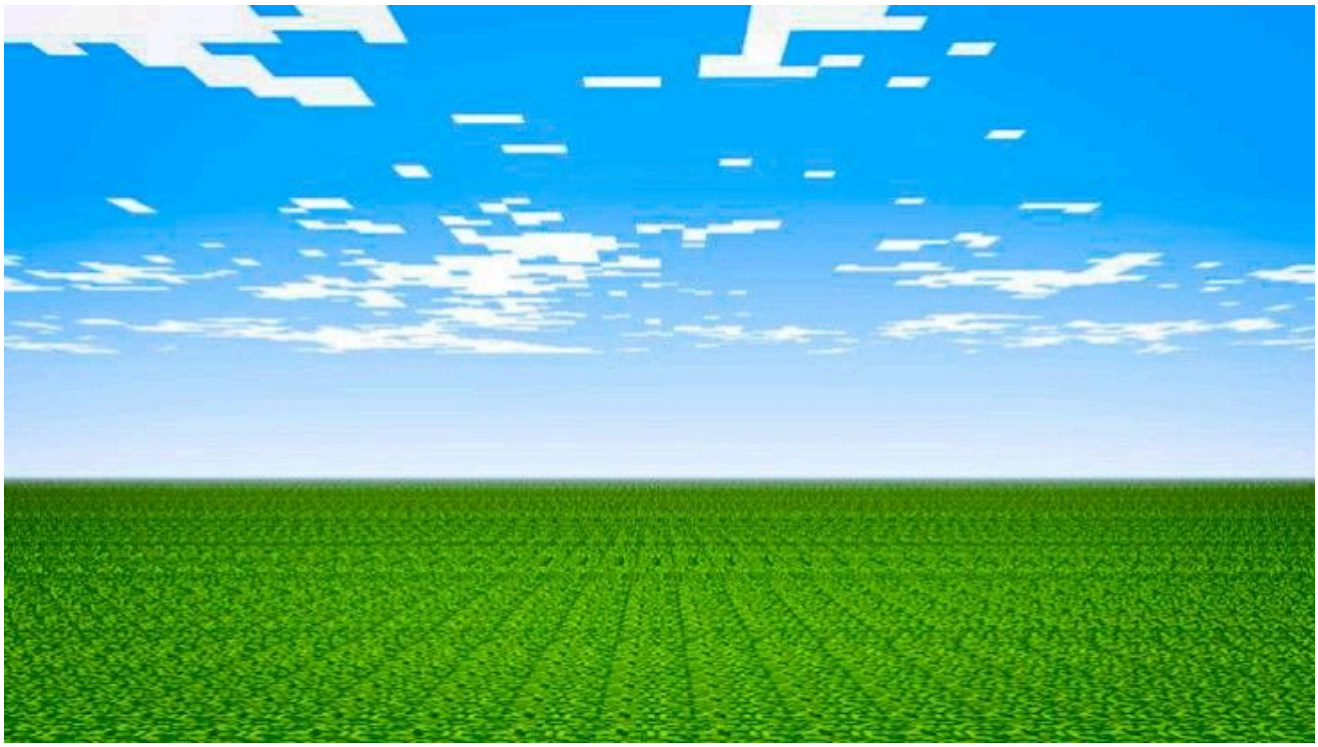
He creado dibujando todos los sprites menos el castillo, es decir, he creado las plataformas, los ítems y el personaje por partes. Los he dibujado en piskel dando como resultado los siguientes sprites (son 2 porque al final decidí añadir otro ítem, la tnt).

Descargado de internet:



Creación de los fondos.

Una vez elegido los sprites, busco los fondos de pantalla tanto el principal como el secundario que hará luego el efecto de movimiento. Estos fondos los he obtenido de internet.



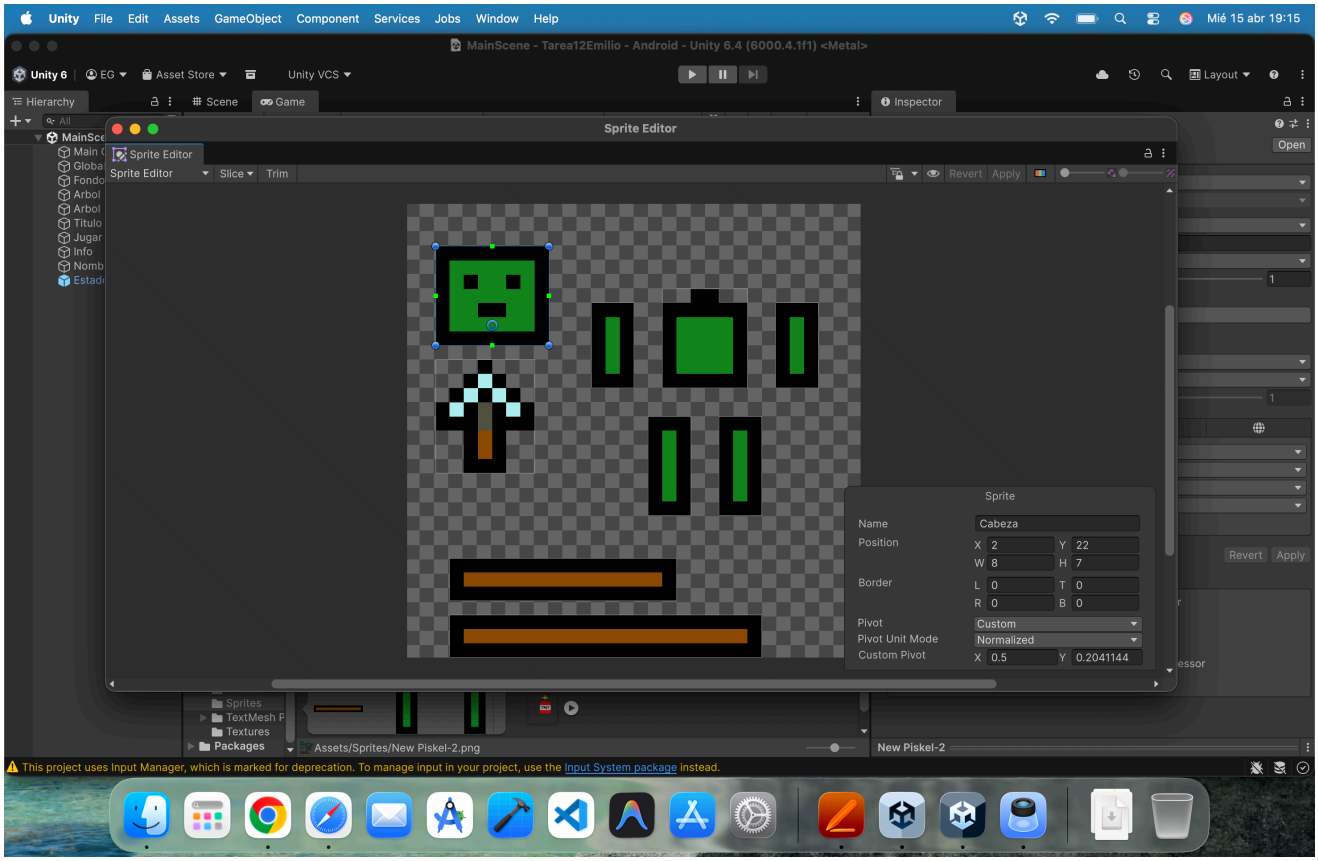
Creación del proyecto

Una vez realizados los sprites y obtenidos los fondos, creo un nuevo proyecto en unity llamado Tarea12Emilio.

Una vez cargado como dicen las indicaciones añado el script AspectUtility.cs a la cámara principal.

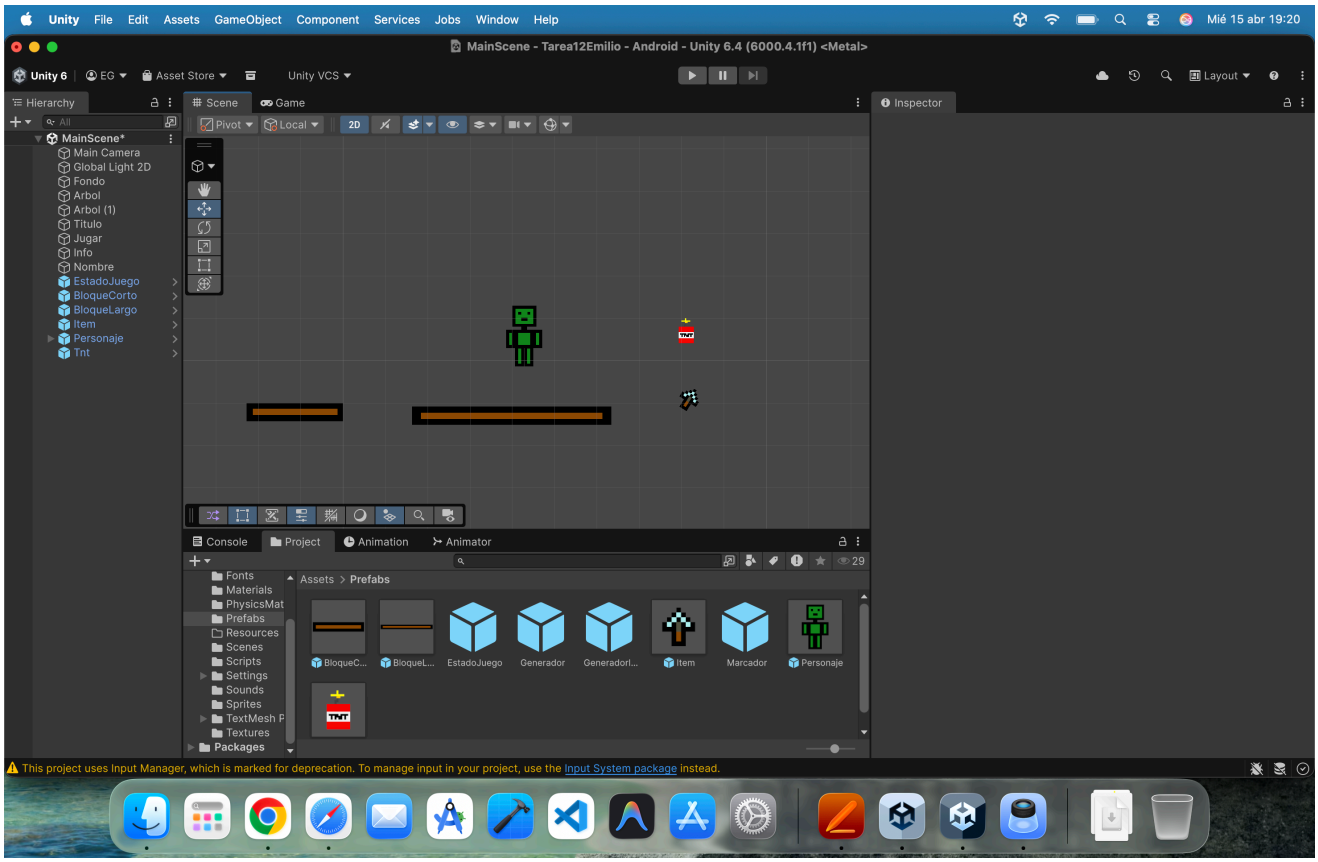
Creo la carpeta de sprites e importo los tres sprites y la carpeta Textures y pongo allí los fondos.

Una vez teniendo los sprites en su carpeta procedo a editarlos para poder recortar y poner los puntos de rotación de cada uno de ellos donde yo elija. Antes de ello para que los sprites se vieran con mejor resolución les cambie sus valores de filterMode a la opción pointNoFilter para que se adaptaran a la resolución y empecé a editar los sprites



Una vez editados y renombrados todos los sprites. Creo los prefabs donde van a ir incluidos para luego poder ser reutilizados. Creo un prefab para ítem, tnt, bloque corto, bloque largo y personaje.

Y añado cada objeto en su prefab una vez editado al tamaño y forma que necesito. El personaje lo voy ensamblando por partes (cuerpo, cabeza, brazo...) hasta que me convence su forma y lo incluyo en su prefab.



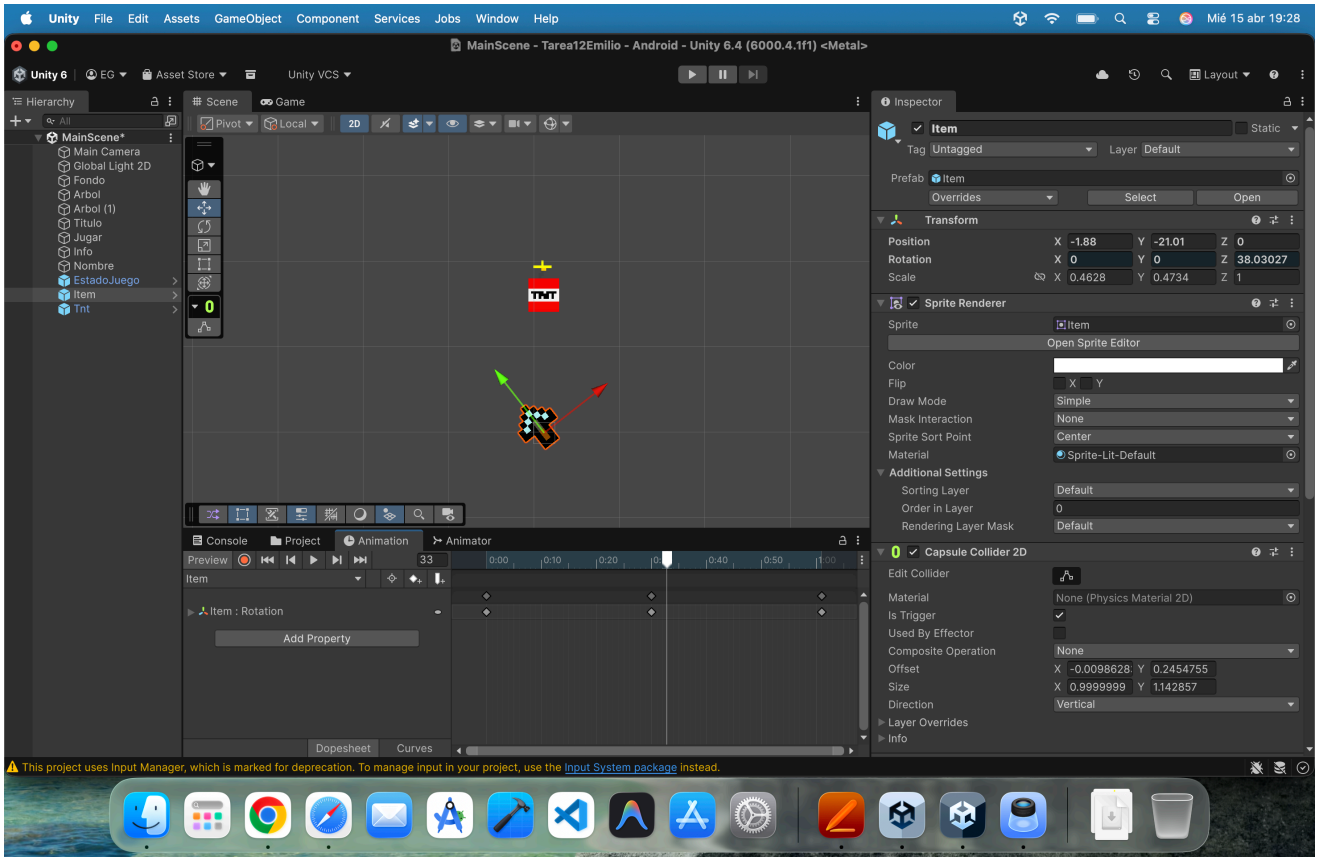
Para las creaciones de los prefabs he seguido la guía de ejemplo del mapache añadiendo colliders al cuerpo, cabeza y piernas del personaje, así como a los bloques e ítems.

Creando animaciones.

He añadido animaciones de movimiento a los ítems y sobre todo al personaje.

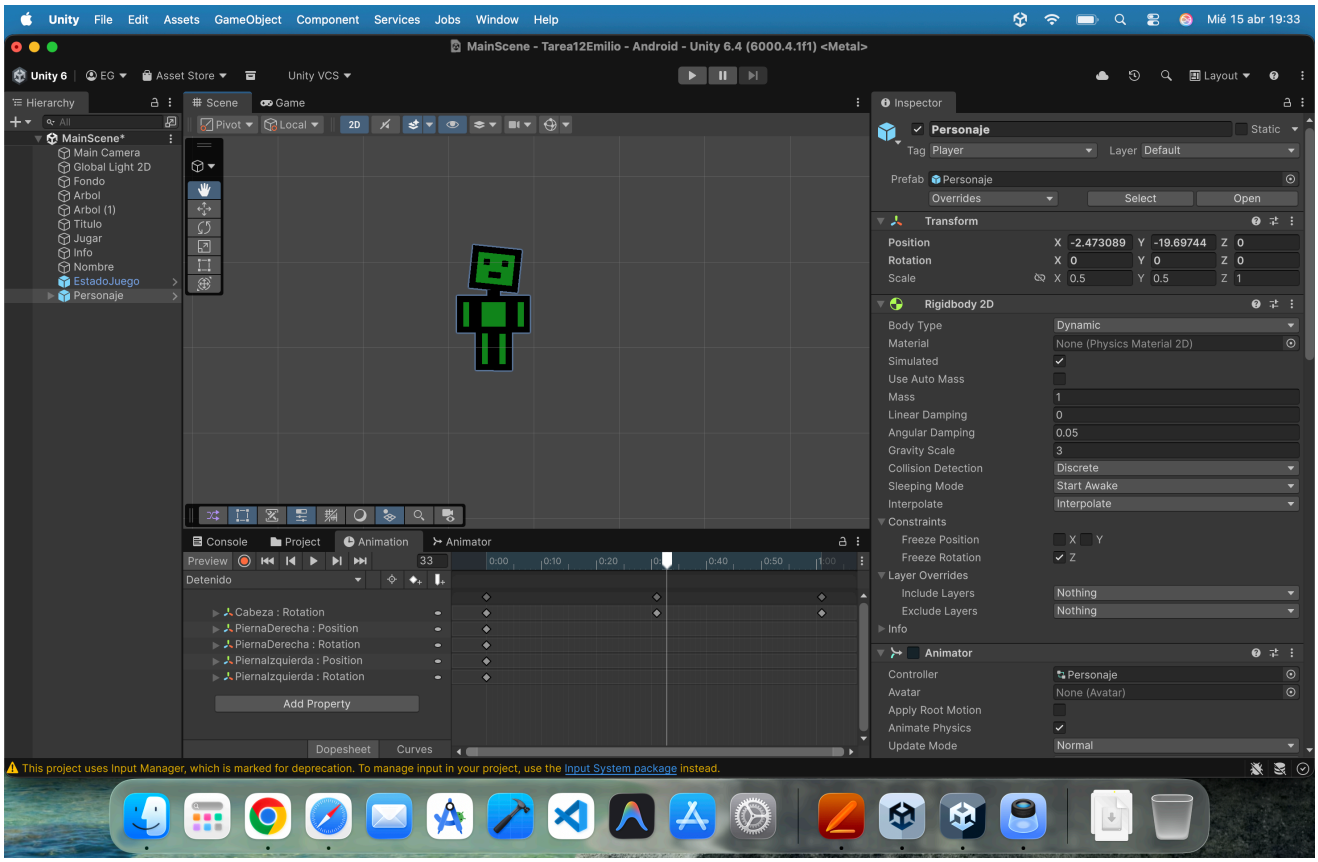
Para los ítems ha sido sencillo, la misma animación para los dos ítems que rotan sobre su punto de giro unos grados a la derecha y otros a la izquierda. Lo he hecho añadiendo un animator al ítem y grabando los estados iniciales y finales quedando un movimiento al estilo de campana. Y ese animator lo he usado para los dos ítems, tanto el pico como el tnt. Y diras porque un tnt también, pues se me ocurrió la idea de que si ya hay algo que suma puntos porque no crear algo

para que los reste, en Minecraft existe el tnt que obviamente te quita vida si estas cerca, pues en este caso resta puntos de los que llevamos conseguidos. El comportamiento en si es igual que el del ítem nada mas que editando los puntos ganados a un numero negativo, concretamente resta un punto.

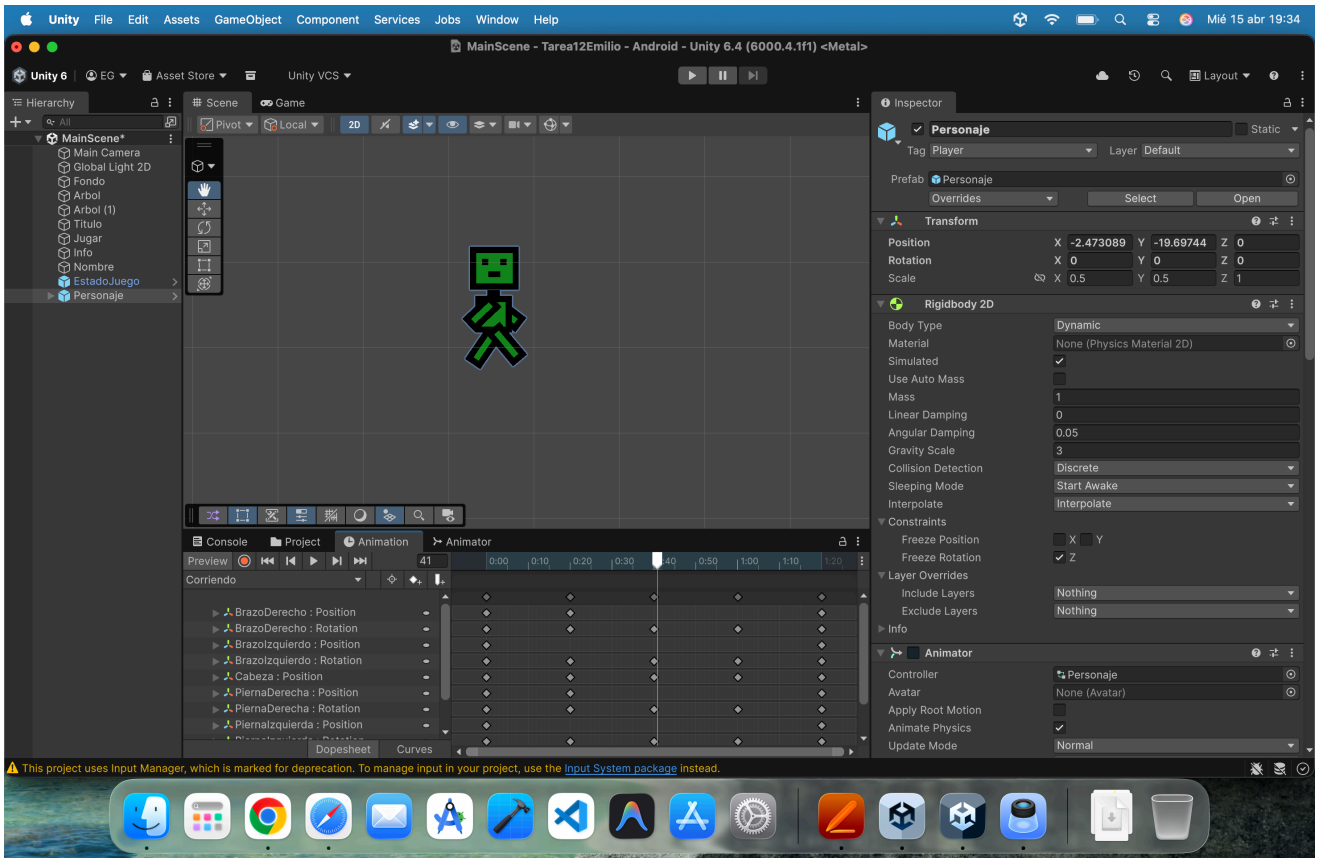


Para el personaje pues he realizado las 3 animaciones que son como las del ejemplo, detenido, corriendo y saltando. He de decir que me costo mucho hacer lo que es la transición de por ejemplo en movimiento a saltando, pues no lo hace automáticamente el cambio de animación tarda y bastante, cosa que no me agrada y he tenido que mirar y buscar como hacerlo y poder ajustar lo máximo que he podido y sabido las transiciones de unos a otros. Para que me entiendas de estar en parado como empieza el personaje al corriendo le daba tiempo a moverse en la animación de parado toda la plataforma hasta que empezaba a animar el corriendo... para mi un desastre, aunque he conseguido ajustarla de la manera que puedes ajustar el tiempo o forma de transición de una animación a otra desde el animator.

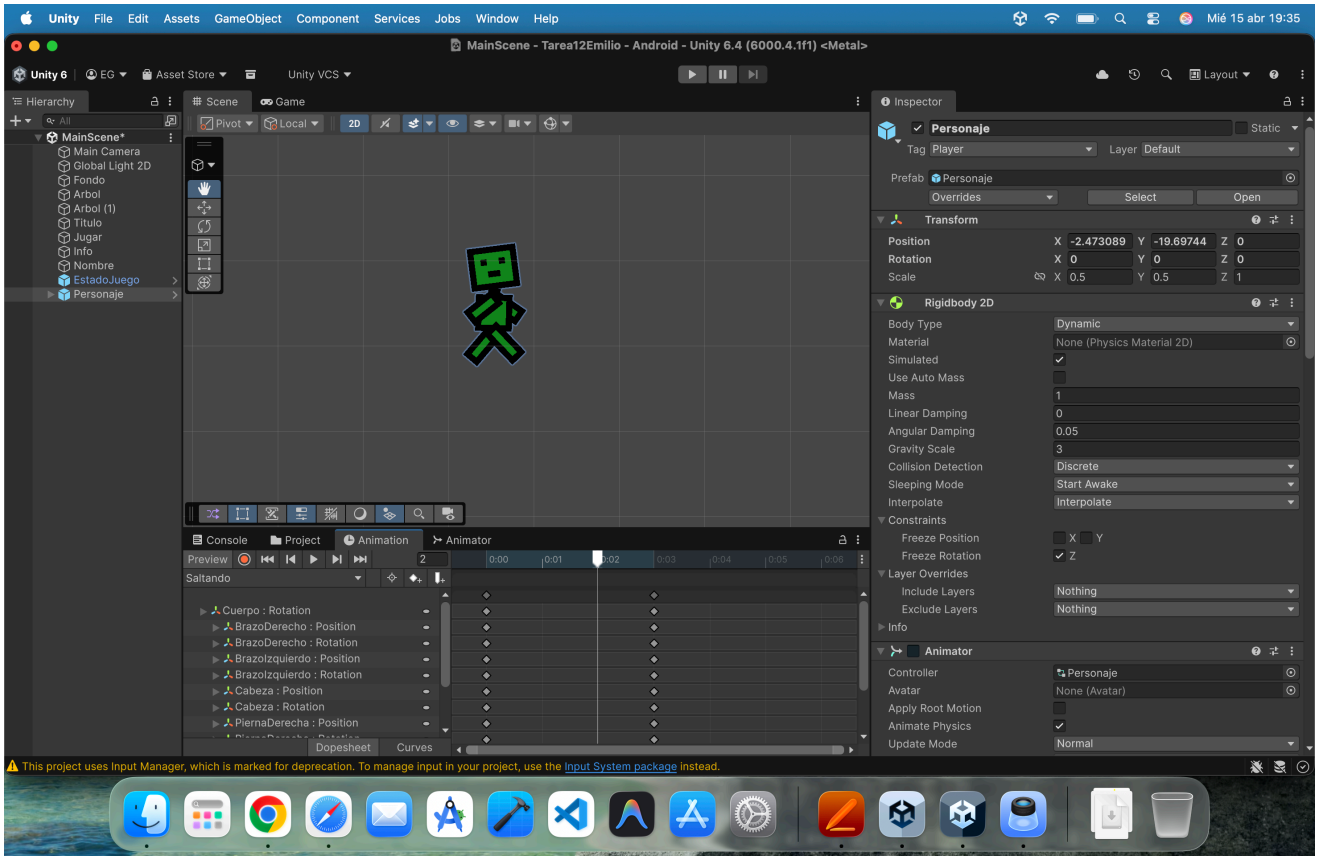
Detenido: moco mueve la cabeza de un lado a otro.



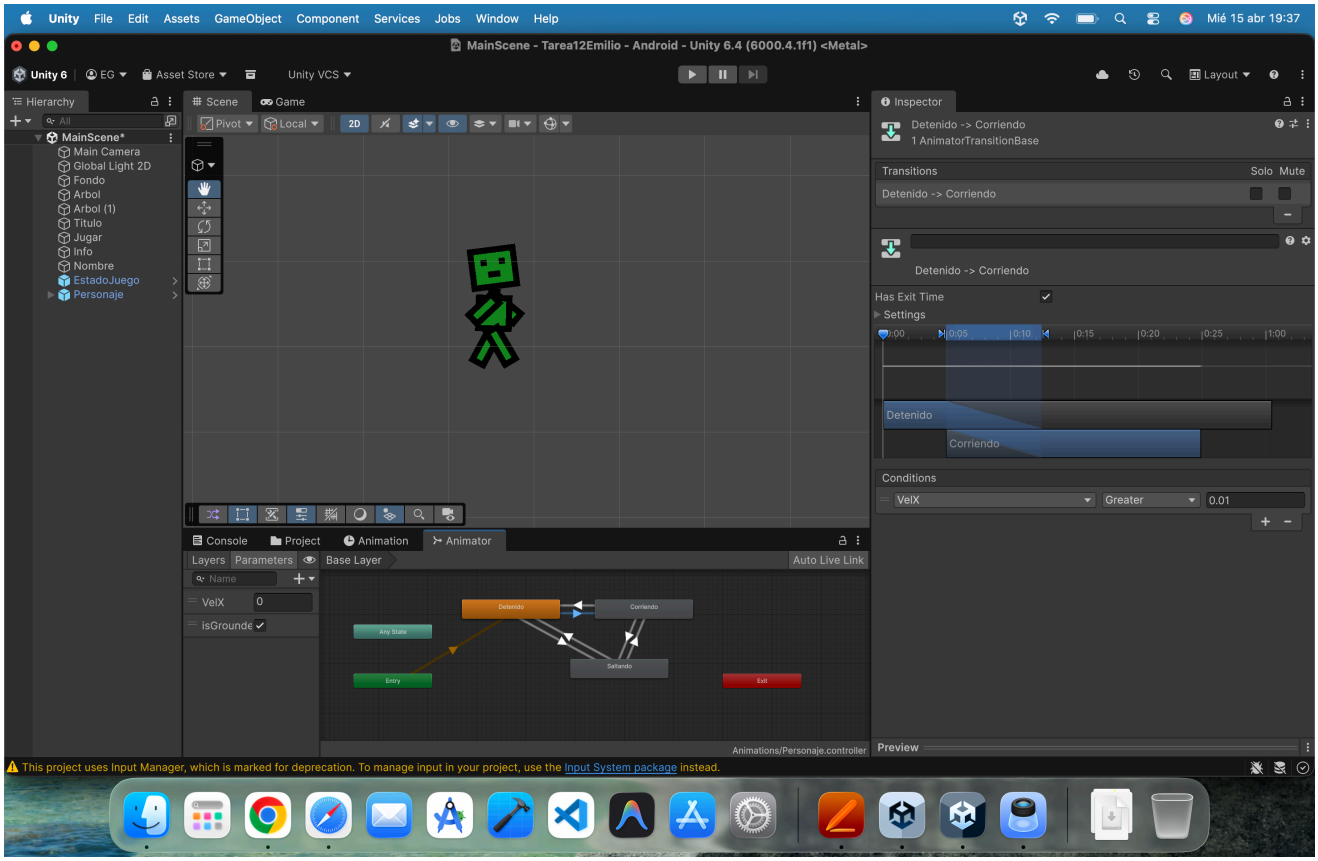
Corriendo: moco mueve las piernas y los brazos como sería la forma de correr y girando su cuerpo para ello.



Saltando: moco mueve las piernas en estado abierto en repetidas ocasiones.



Paso de transiciones de unas a otras añadiendo las variables VelX y isGrouded para saber si esta en movimiento y tocando el suelo.

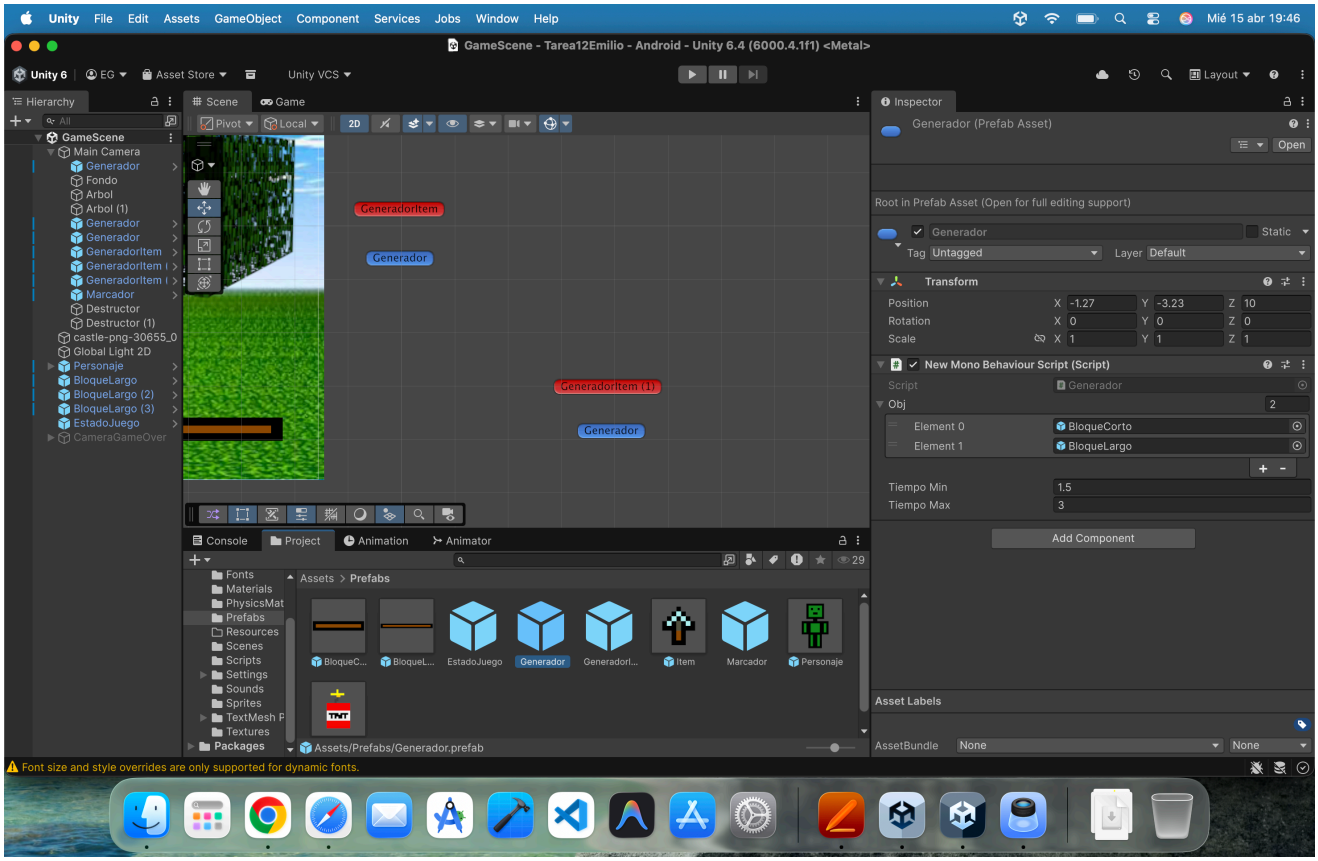


Generadores de bloques e items y destructores.

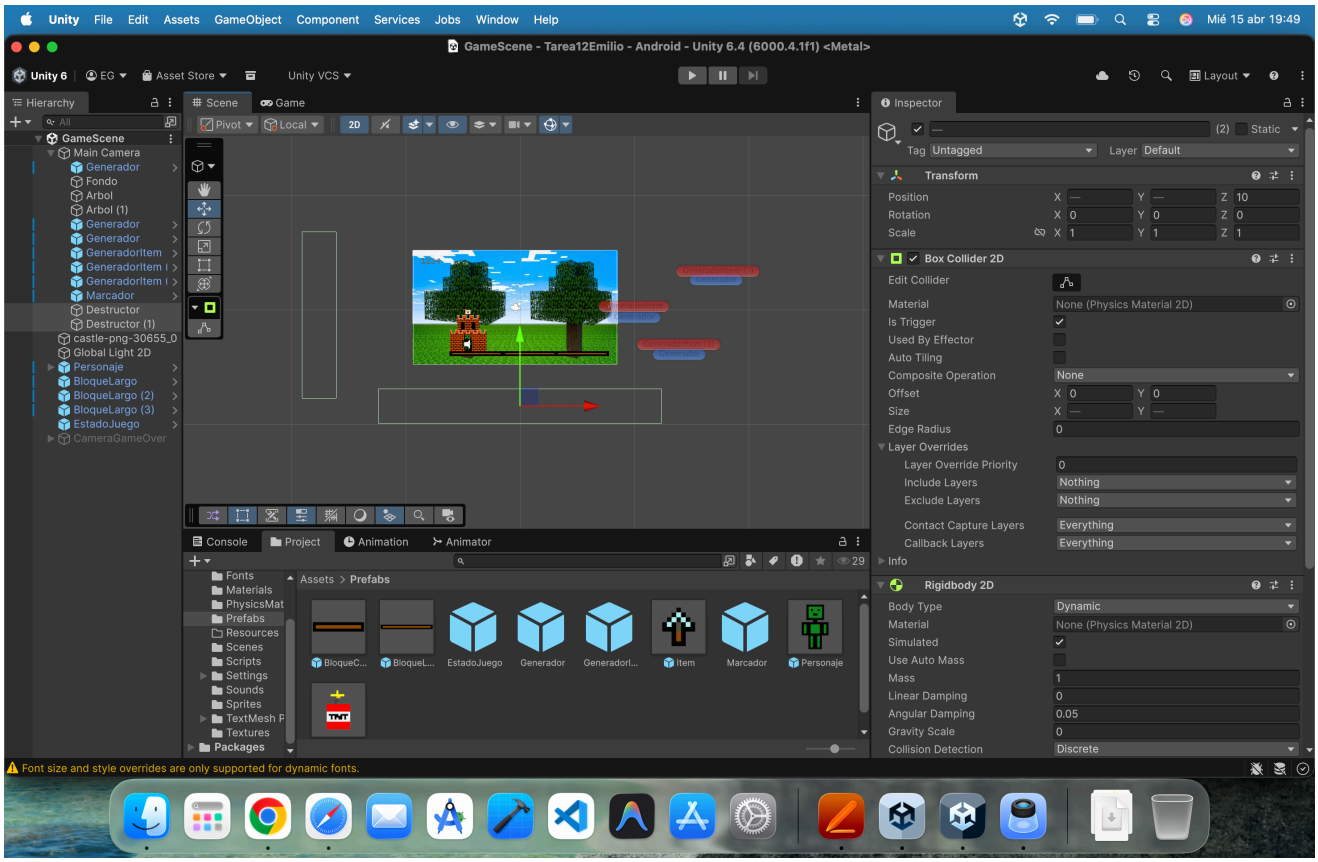
Para que el juego vaya generando de forma aleatoria tanto bloques como ítems he creado un generador para cada uno de ellos y en cada altura (3 alturas he querido para el juego) para que se vayan generando.

Para ello he credo un prefab para los bloques y otro para los ítems. El comportamiento es el mismo para ambos pero irán generando una cosa u otra, todo mediante código que luego al final añadiré todos los códigos utilizados, pero en si son los mismos que los de los videos explicando el corre corre del mapache.

En el prefab que hemos creado a raíz de un gameobject vacío le añadimos el script y en el podemos añadir lo que queremos que aparezca y el tiempo máximo y mínimo de generación de cada cosa (bloques o ítems).



Para los destructores, haremos otro gameObject en el cual le incorporamos el script destructor, que se encargara de destruir los bloques e ítems que se hayan generado y pasado por la pantalla, así como, al personaje cuando cae y sale de la pantalla significando el game over o muerte del personaje. Uno se encuentra a la parte izquierda de la pantalla y el otro en la parte de abajo de la pantalla, todo ello fuera de cámara para que no se vea.



Fondos efecto paralax:

Para la creación de los fondos con efecto de movimiento o paralax, he añadido el fondo en la parte de atrás y el árbol lo he duplicado haciendo que cada uno se “superponga” al moverse haciendo la sensación de que es una hilera de árboles uno tras otro. Para ello se añade un quad a un gameobject y luego mediante el script scroll se le da el movimiento eligiendo una velocidad adecuada al avance del juego y parando cuando acabe o iniciando cuando inicie el juego. También he añadido el marcador para que sea visible la puntuación que llevamos durante el juego.

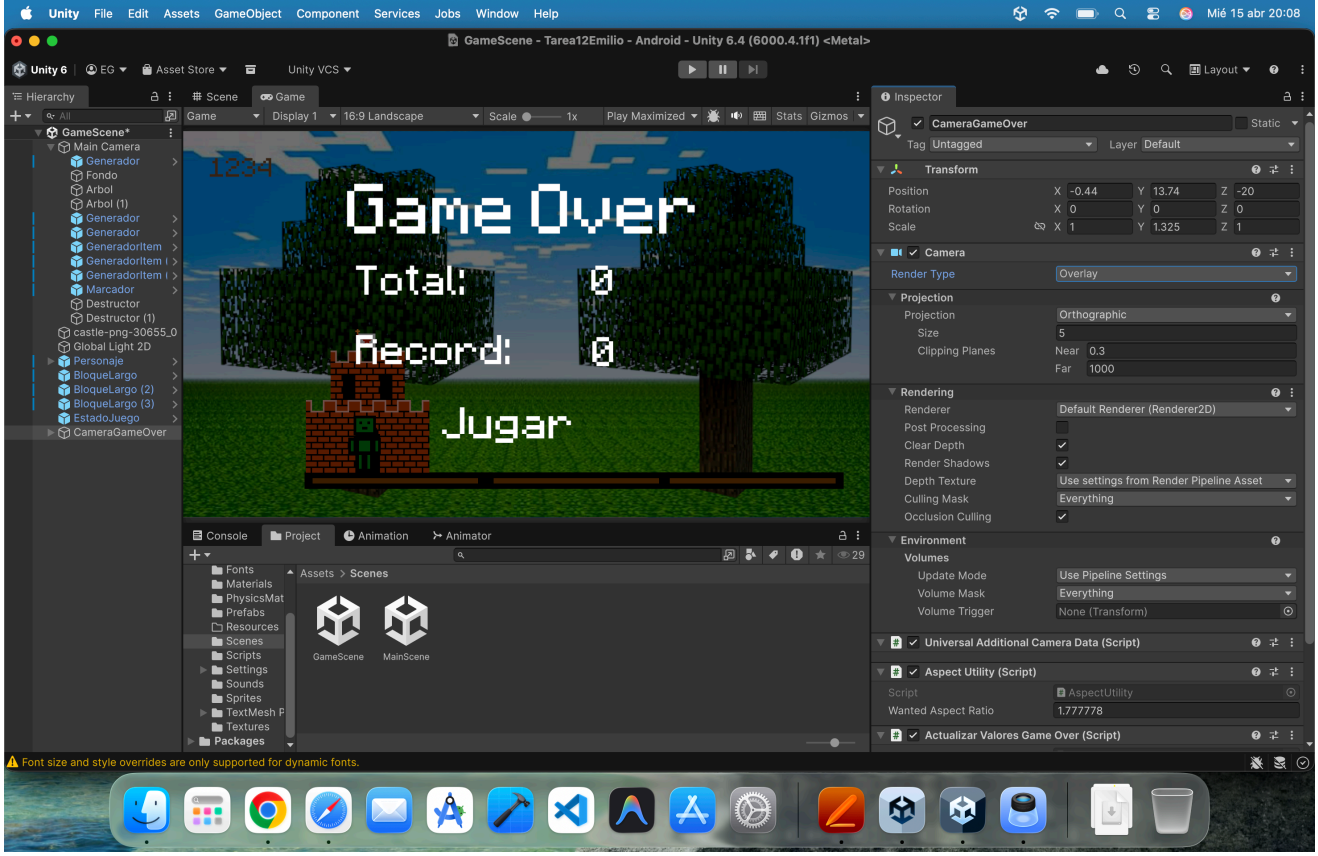
Aqui me encontre con otro problema pues los shader para hacer el material necesario para dar la sensación del movimiento no son para nada lo que se explica en la guía ni en el video del mapache, he tenido que buscar y poder adecuarlo a mi proyecto, creo haber elegido las adecuadas y para mi gusto de tantas las que he probado creo que es la que mejor queda. En mi caso han sido unli/transparent sobre todo para los arboles

Creación de la pantalla principal y Game Over.

Para la pantalla principal he cogido los dos fondos en estático y he añadido las letras y el “botón” jugar mediante un collider.

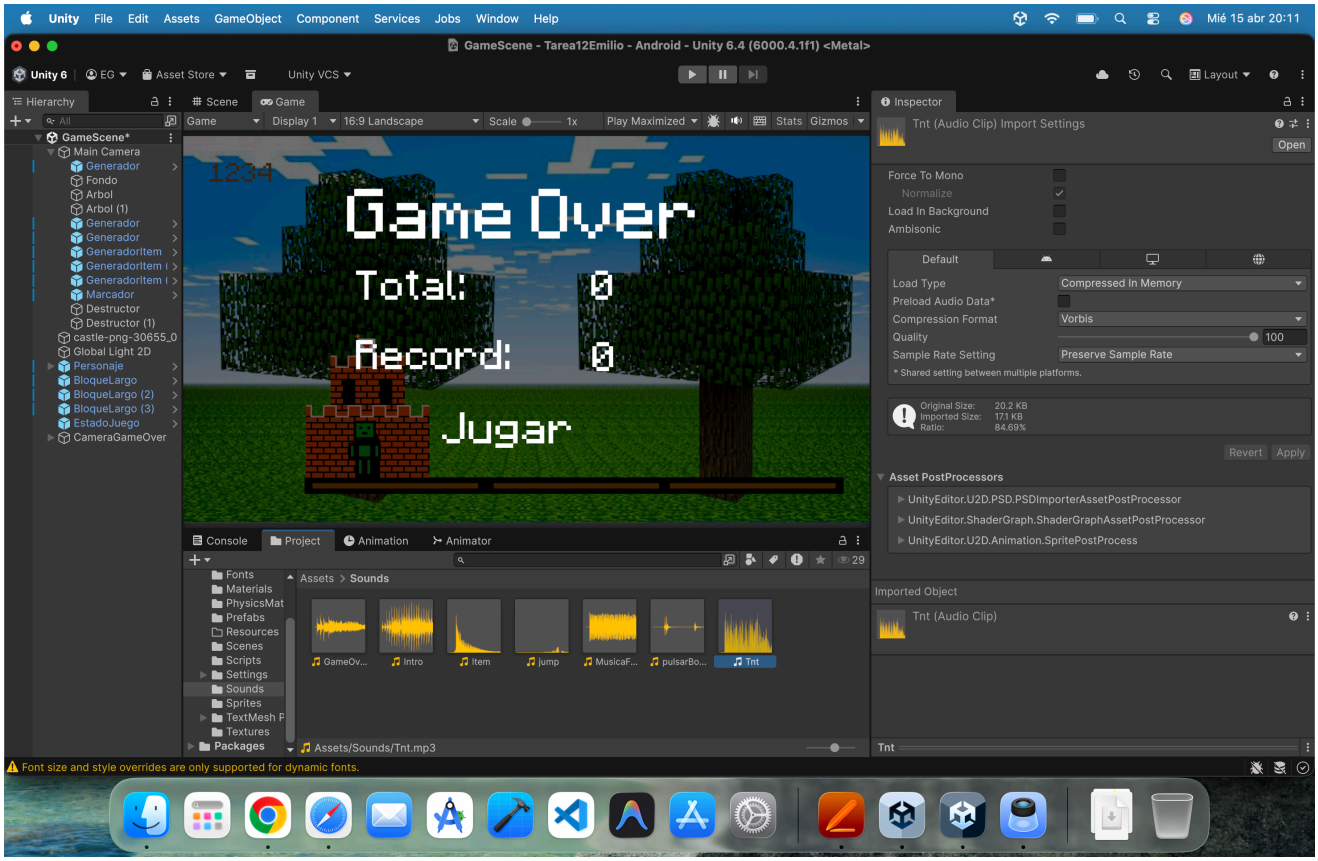
Para poder crear esta pantalla he tenido que hacer otra escena del juego para que la incluya, luego mediante código cambiamos de una pantalla a otra pulsando el botón jugar y guardando el estado de juego en un gameobject para no tener problemas al cambio de una pantalla a otra y mantener datos entre una y otra.

Para la pantalla de gameOver no he creado otra escena sino otra cámara en la escena del juego, para que cuando el personaje muera se opaque la pantalla y salga el game over con la puntuación del juego, la del récord y el botón jugar para iniciar una nueva partida. Aquí he tenido mucho problema pues no conseguía hacer que la pantalla se quedara opaca sino que salía en un color solo uniforme, todo era debido a que hay que cambiar el render type a overlay, y luego hay que hacer como dice la tarea poner orthographic y en la cámara principal hay que poner en Stack como hija la gameover para que haga su función y salga como se pide de forma que tenga una opacidad baja o semi transparencia. Para mi un quebradero de cabeza la verdad.



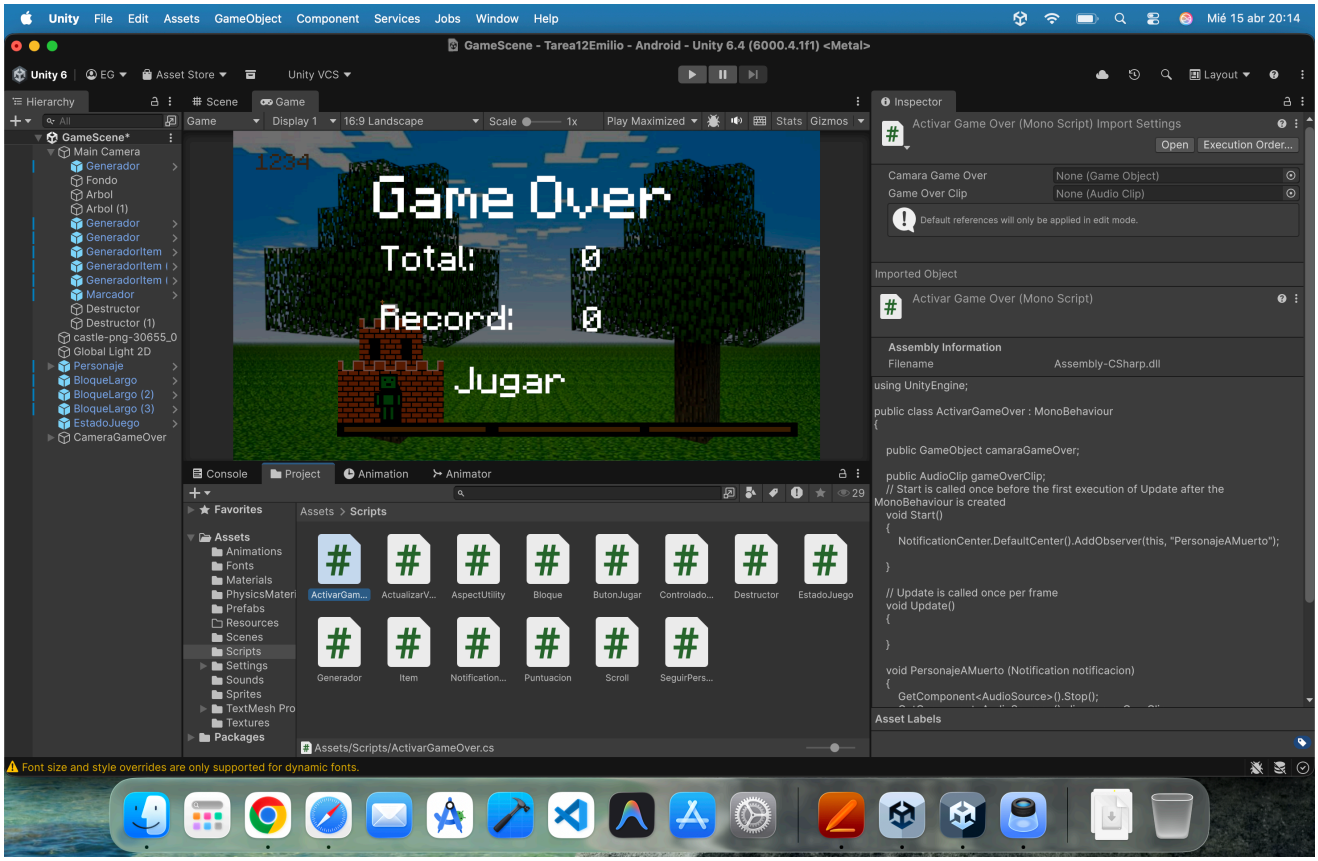
Añadir sonidos.

Esto ha sido algo sencillo he añadido una carpeta llamada sounds, donde he introducido todos los sonidos que me han parecido para el juego obtenidos de internet. En si sonara una musica en la pantalla de bienvenida y otra durante el juego, al pulsar al botón jugar, cada vez que se salta, que se coge un ítem (pico), cuando se coge una tnt, y cuando se muere en la pantalla de gameOver otra canción.



Todos los Scripts (códigos).

Todos los scripts utilizados son como los del ejemplo del mapache, en algunos he variado la velocidad o tiempos de por ejemplo aparecer ítems pero en si son todos los mismos. Los incorporo al archivo.



ACTIVARGAMEOVER.CS

```
using UnityEngine;

public class ActivarGameOver : MonoBehaviour
{

    public GameObject camaraGameOver;

    //variable para manejar el sonido de gameover
    public AudioClip gameOverClip;
    // Start is called once before the first execution of Update
after the MonoBehaviour is created
    void Start()
    {
        //Mes suscribo a la notificación para saber si ha muerto el
personaje
        NotificationCenter.DefaultCenter().AddObserver(this,
"PersonajeAMuerto");

    }

    // Update is called once per frame
    void Update()
    {
```

```
    }

    void PersonajeAMuerto (Notification notificacion)
    {
        // si el personaje muere, paro la musica del juego,
reproduzco la de game over y muestro la cámara de game over.
        GetComponent<AudioSource>().Stop();
        GetComponent<AudioSource>().clip = gameOverClip;
        GetComponent<AudioSource>().loop = false;
        GetComponent<AudioSource>().Play();
        camaraGameOver.SetActive(true);

    }
}
```

ACTUALIZARVALORESGAMEOVER.CS

```
using UnityEngine;

public class ActualizarValoresGameOver : MonoBehaviour
{
    //variables para manejar las puntuaciones

    public TextMesh total;
    public TextMesh record;
    public Puntuacion puntuacion;

    // Start is called once before the first execution of Update
after the MonoBehaviour is created
    void Start()
    {

    }

    // Update is called once per frame
    void Update()
    {

    }
    //se activa sólo cuando la cámara game over este activa
    void OnEnable()
    {
        //se actualizan los textos con las puntuaciones
        total.text = puntuacion.puntuacion.ToString();
        if(EstadoJuego.estadoJuego != null){
            record.text =
EstadoJuego.estadoJuego.puntuacioinMaxima.ToString();
```

```
        }
    }
}
```

BLOQUE.CS

```
using UnityEngine;

public class Bloque : MonoBehaviour
{
    //variables para la puntuación y saber si han colisionado o no
el personaje
    private bool haColisionadoConPlayer = false;
    public int puntosGanados = 1;


    // Start is called once before the first execution of Update
after the MonoBehaviour is created
    void Start()
    {

    }

    // Update is called once per frame
    void Update()
    {

    }

    //si han colisionado con el bloque (con su collider) se incia
el método
    void OnCollisionEnter2D(Collision2D collision)
    {
        // si el jugador ha colisionado cno la pierna izquierda
(para evitar doble punto)
        if (!haColisionadoConPlayer && collision.gameObject.tag ==
"Player")
        {
            GameObject obj =
collision.contacts[0].collider.gameObject;
            if(obj.name == "PiernaIzquierda")
            {
                haColisionadoConPlayer=true;
                //incrementamos los puntos
```

```
NotificationCenter.DefaultCenter().PostNotification(this,
"IncrementarPuntos", puntosGanados);

            }

        }

    }
}
```

BUTONJUGAR.CS

```
using UnityEngine;

public class BuotonJugar : MonoBehaviour
{
    // Start is called once before the first execution of Update
after the MonoBehaviour is created
    void Start()
    {

    }

    // Update is called once per frame
    void Update()
    {

    }

    //este método se activa si se hace clic en el collider de jugar
    void OnMouseDown()
    {
        //se detiene el audio de la pantalla de inicio y se inicia
el audio del juego después de la duración del audio del boton jugar
        Camera.main.GetComponent<AudioSource>().Stop();
        GetComponent<AudioSource>().Play();
        //se llama al método para iniciar la pantalla del juego
        Invoke("CargarNivelJuego",
GetComponent<AudioSource>().clip.length);

    }

    //método que inicia el juego, mas bien la scene donde esta el
juego
    void CargarNivelJuego()
```

```
    {
      Application.LoadLevel("GameScene");

    }
}
```

CONTROLADORPERSONAJE.CS

```
using UnityEngine;

public class ControladorPersonaje : MonoBehaviour
{
    //variables para manejar el personaje
    public float fuerzaSalto = 100f;
    private bool enSuelo = true;
    public Transform comprobadorSuelo;
    private float comprobadorRadio = 0.07f;
    public LayerMask mascaraSuelo;
    private bool dobleSalto = false;
    private Animator animator;
    private bool corriendo = false;
    public float velocidad = 1f;

    //se llama antes del start y asi se llama siempre a las
animaciones
    void Awake()
    {
        animator = GetComponent<Animator>();
    }

    // Start is called once before the first execution of Update
after the MonoBehaviour is created
    void Start()
    {

    }

    void FixedUpdate()
    {
        //control del personaje si esta corriendo y en el suelo,
asi como el doble salto
        if (corriendo)
        {
            GetComponent<Rigidbody2D>().linearVelocity = new
Vector2(velocidad, GetComponent<Rigidbody2D>().linearVelocity.y);
        }
```

```
        animator.SetFloat("VelX",
GetComponent<Rigidbody2D>().linearVelocity.x);
        enSuelo =
Physics2D.OverlapCircle(comprobadorSuelo.position,
comprobadorRadio, mascaraSuelo);
        animator.SetBool("isGrounded", enSuelo);

        if (enSuelo)
        {
            dobleSalto = false;
        }
    }

    // Update is called once per frame
    void Update()
    {
        //funcion para si se hace clic el personaje salte y
controla tambien el doble salto. si estaba parado empieza a correr
        if (Input.GetMouseButtonDown(0))
        {
            if (corriendo)
            {
                if (enSuelo || !dobleSalto)
                        {
                            GetComponent<AudioSource>().Play();

GetComponent<Rigidbody2D>().linearVelocity = new
Vector2(GetComponent<Rigidbody2D>().linearVelocity.x, fuerzaSalto);

GetComponent<Rigidbody2D>().AddForce(new Vector2(0,fuerzaSalto));
                            if (!dobleSalto && !enSuelo)
                            {
                                dobleSalto = true;
                            }

                        }
            }
            else
            {
                corriendo = true;

NotificationCenter.DefaultCenter().PostNotification(this,
"PersonajeEmpiezaACorrer");
            }
        }



    }
}
```

DESTRUCTOR.CS

```
using UnityEngine;

public class Destructor : MonoBehaviour
{
    // Start is called once before the first execution of Update
after the MonoBehaviour is created
    void Start()
    {

    }

    // Update is called once per frame
    void Update()
    {

    }

    //método para que cuando un collider toque al collider del
destructor este haga que se detruya y si es el personaje cree la
notificación de que ha muerto
    void OnTriggerEnter2D(Collider2D other){

        if(other.tag == "Player")
        {

NotificationCenter.DefaultCenter().PostNotification(this,
"PersonajeAMuerto");
            GameObject personaje = GameObject.Find("Personaje");
            personaje.SetActive(false);
        }else{
            Destroy(other.gameObject);
        }

    }
}
```

ESTADODEJUEGO.CS

```
using UnityEngine;
using System;
using System.Runtime.Serialization.Formatters.Binary;
using System.IO;
```

```
public class EstadoJuego : MonoBehaviour
{
    public int puntuacioinMaxima;
    public static EstadoJuego estadoJuego;
    private string rutaArchivo;

    //se llama al estado del juego para que no se destruya si ya
existe.
    void Awake()
    {
        rutaArchivo = Application.persistentDataPath + "/
datos.dat";
        if (estadoJuego == null)
        {
            estadoJuego = this;
            DontDestroyOnLoad (gameObject);
        } else if (estadoJuego != this)
        {
            Destroy(gameObject);
        }
    }
    // Start is called once before the first execution of Update
after the MonoBehaviour is created
    void Start()
    {
        Cargar();
    }

    // Update is called once per frame
    void Update()
    {

    }

    //método para cargar el archivo d eestado de juego, si no
existe se carga 0 en la puntuación máxima
    void Cargar()
    {

        if(File.Exists(rutaArchivo)){
            BinaryFormatter bf = new BinaryFormatter();
            FileStream file = File.Open(rutaArchivo,
FileMode.Open);

            DatosAGuardar datos = (DatosAGuardar)
bf.Deserialize(file);
            puntuacioinMaxima = datos.puntuacioinMaxima;

            file.Close();
        }else
        {
```

```
            puntuacioinMaxima = 0;
        }

    }

    //método para guardar el estado del juego
    public void Guardar()
    {
        BinaryFormatter bf = new BinaryFormatter();
        FileStream file = File.Create(rutaArchivo);

        DatosAGuardar datos = new DatosAGuardar();
        datos.puntuacioinMaxima = puntuacioinMaxima;

        bf.Serialize(file, datos);

        file.Close();
    }

}

//clase de los datos a guardar, solo guardamos la puntuación
máximas
[Serializable]
class DatosAGuardar
{
    public int puntuacioinMaxima;


}
```

GENERADOR.CS

```
using UnityEngine;

public class NewMonoBehaviourScript : MonoBehaviour
{
    //variables para generar objetos segun un tiempo mínimo y
máximo

    public GameObject[] obj;
    public float tiempoMin = 1.5f;
    public float tiempoMax = 3f;
    private bool fin = false;

    // Start is called once before the first execution of Update
after the MonoBehaviour is created
    void Start()
    {
```

```
        //estamos pendientes de las notificaciones para saber
cuando iniciar o detener la generación de objetos

        NotificationCenter.DefaultCenter().AddObserver(this,
"PersonajeEmpiezaACorrer");
        NotificationCenter.DefaultCenter().AddObserver(this,
"PersonajeAMuerto");
}

    void PersonajeEmpiezaACorrer(Notification notification)
    {
        Generar();

    }

    // Update is called once per frame
    void Update()
    {

    }

    // método para generar los objetos de forma aleatoria y por el
tiempo establecido
    void Generar()
    {
        if(!fin){
            Instantiate(obj[Random.Range(0, obj.Length)],
transform.position, Quaternion.identity);
            Invoke("Generar", Random.Range(tiempoMin, tiempoMax));
        }
    }

    //si el personaje muere y nos lo dice la notificación se
detiene la generación de objetos.
    void PersonajeAMuerto()
    {
        fin = true;
    }
}
```

ITEM.CS

```
using UnityEngine;

public class Item : MonoBehaviour
{
```

```
    //variables de los puntos obtenidos si cogemos el item como de
su sonido
    public int puntosGanados = 5;
    public AudioClip itemSonido;
    public float itemSoundVolume = 1f;

    // Start is called once before the first execution of Update
after the MonoBehaviour is created
    void Start()
    {

    }

    // Update is called once per frame
    void Update()
    {

    }

    //si el jugador choca con el item sumamos los puntos y
reproducimos su sonido, también se destruye el item
    void OnTriggerEnter2D(Collider2D collider)
    {
        if(collider.tag == "Player")
        {

NotificationCenter.DefaultCenter().PostNotification(this,
"IncrementarPuntos", puntosGanados);
            AudioSource.PlayClipAtPoint(itemSonido,
Camera.main.transform.position, itemSoundVolume);
        }
        Destroy(gameObject);
    }
}
```

NOTIFICATIONCENTER.CS

```
// Más información sobre este script aquí:
// http://wiki.unity3d.com/index.php?
title=NotificationCenterGenerics

using UnityEngine;
using System.Collections;
using System.Collections.Generic;
using System;

//    NotificationCenter is used for handling messages between
GameObjects.
```

```
//    GameObjects can register to receive specific notifications.
When another objects sends a notification of that type, all
GameObjects that registered for it and implement the appropriate
message will receive that notification.
//    Observing GameObjects must register to receive notifications
with the AddObserver function, and pass their selves, and the name
of the notification.  Observing GameObjects can also unregister
themselves with the RemoveObserver function.  GameObjects must
request to receive and remove notification types on a type by type
basis.
//    Posting notifications is done by creating a Notification
object and passing it to PostNotification.  All receiving
GameObjects will accept that Notification object.  The Notification
object contains the sender, the notification type name, and an
option hashtable containing data.
//    To use NotificationCenter, either create and manage a unique
instance of it somewhere, or use the static NotificationCenter.

// We need a static method for objects to be able to obtain the
default notification center.
// This default center is what all objects will use for most
notifications.  We can of course create our own separate instances
of NotificationCenter, but this is the static one used by all.
public class NotificationCenter : MonoBehaviour
{
    private static NotificationCenter defaultCenter;
    public static NotificationCenter DefaultCenter () {
        // If the defaultCenter doesn't already exist, we need to
create it
        if (!defaultCenter) {
            // Because the NotificationCenter is a component, we
have to create a GameObject to attach it to.
            GameObject notificationObject = new GameObject("Default
Notification Center");
            // Add the NotificationCenter component, and set it as
the defaultCenter
            defaultCenter =
notificationObject.AddComponent<NotificationCenter>();
            DontDestroyOnLoad(defaultCenter);
        }

        return defaultCenter;
    }

    // Our hashtable containing all the notifications.  Each
notification in the hash table is an ArrayList that contains all
the observers for that notification.
    Hashtable notifications = new Hashtable();

    // AddObserver includes a version where the observer can
request to only receive notifications from a specific object.  We
```

```
haven't implemented that yet, so the sender value is ignored for
now.
    public void AddObserver (Component observer, String name)
{ AddObserver(observer, name, null); }
    public void AddObserver (Component observer, String name,
object sender) {
        // If the name isn't good, then throw an error and return.
        if (name == null || name == "") { Debug.Log("Null name
specified for notification in AddObserver."); return; }
        // If this specific notification doens't exist yet, then
create it.
        if (!notifications.ContainsKey(name)) {
            notifications[name] = new List<Component>();
        }
//      if (!notifications[name]) {
//          notifications[name] = new List<Component>();
//      }

        List<Component> notifyList =
(List<Component>)notifications[name];

        // If the list of observers doesn't already contain the one
that's registering, then add it.
        if (!notifyList.Contains(observer))
{ notifyList.Add(observer); }
    }

    // RemoveObserver removes the observer from the notification
list for the specified notification type
    public void RemoveObserver (Component observer, String name) {
        List<Component> notifyList =
(List<Component>)notifications[name]; //change from original

        // Assuming that this is a valid notification type, remove
the observer from the list.
        // If the list of observers is now empty, then remove that
notification type from the notifications hash.  This is for
housekeeping purposes.
        if (notifyList != null) {
            if (notifyList.Contains(observer))
{ notifyList.Remove(observer); }
            if (notifyList.Count == 0)
{ notifications.Remove(name); }
        }
    }

    // PostNotification sends a notification object to all objects
that have requested to receive this type of notification.
    // A notification can either be posted with a notification
object or by just sending the individual components.
```

```
    public void PostNotification (Component aSender, String aName)
{ PostNotification(aSender, aName, null); }
    public void PostNotification (Component aSender, String aName,
object aData) { PostNotification(new Notification(aSender, aName,
aData)); }
    public void PostNotification (Notification aNotification) {
        // First make sure that the name of the notification is
valid.
        if (aNotification.name == null || aNotification.name == "")
{ Debug.Log("Null name sent to PostNotification."); return; }
        // Obtain the notification list, and make sure that it is
valid as well
        List<Component> notifyList =
(List<Component>)notifications[aNotification.name]; //change from
original
        if (notifyList == null) { Debug.Log("Notify list not found
in PostNotification."); return; }

        // Clone list, so there won't be an issue if an observer is
added or removed while notifications are being sent
        notifyList = new List<Component>(notifyList);

        // Create an array to keep track of invalid observers that
we need to remove
        List<Component> observersToRemove = new
List<Component>(); //change from original

        // Itterate through all the objects that have signed up to
be notified by this type of notification.
        foreach (Component observer in notifyList) {
            // If the observer isn't valid, then keep track of it
so we can remove it later.
            // We can't remove it right now, or it will mess the
for loop up.
            if (!observer) { observersToRemove.Add(observer);
            } else {
                // If the observer is valid, then send it the
notification.  The message that's sent is the name of the
notification.
                observer.SendMessage(aNotification.name,
aNotification, SendMessageOptions.DontRequireReceiver);
            }
        }

        // Remove all the invalid observers
        foreach (Component observer in observersToRemove) {
            notifyList.Remove(observer);
        }
    }
}
```

```
// The Notification class is the object that is sent to receiving
objects of a notification type.
// This class contains the sending GameObject, the name of the
notification, and optionally a hashtable containing data.
public class Notification {
    public Component sender;
    public String name;
    public object data;

    public Notification (Component aSender, String aName) { sender
= aSender; name = aName; data = null; }
    public Notification (Component aSender, String aName, object
aData) { sender = aSender; name = aName; data = aData; }
}
```

PUNTUACIÓN.CS

```
using UnityEngine;

public class Puntuacion : MonoBehaviour
{

    //variabes para el marcador de puntos
    public int puntuacion = 0;
    public TextMesh marcador;



    // Start is called once before the first execution of Update
after the MonoBehaviour is created
    void Start()
    {
        //vemos las notificaciones que incrementan o paran la suma
de puntos
        NotificationCenter.DefaultCenter().AddObserver(this,
"IncrementarPuntos");
        NotificationCenter.DefaultCenter().AddObserver(this,
"PersonajeAMuerto");
        //actualizamos el marcador con esos puntos
        ActualizarMarcador();
    }

    //método para incrementar puntos mediante la notificación
    void IncrementarPuntos(Notification notificacion)
    {
        //incrementamos los puntos que nos envia la notificación y
actualizamos el marcador con ello
        // por ejemplo si un bloque nos suma 1, nos envia la
notificación que la pierna izquierda choca en el bloque
        // y se obtiene puntos y se suman aqui al marcador.
```

```
        int puntosAIncrementar = (int)notificacion.data;
        puntuacion += puntosAIncrementar;
        ActualizarMarcador();
        Debug.Log("Inrementado" + puntosAIncrementar + "Total "  +
puntuacion);
    }

    // si en la notificación se ha recibido que el personaje ha
muerto, se detiene la suma de puntos y se guardan en el estado de
juego.
    void PersonajeAMuerto(Notification notificacion)
    {
        if(puntuacion > EstadoJuego.estadoJuego.puntuacioinMaxima)
        {
            EstadoJuego.estadoJuego.puntuacioinMaxima = puntuacion;
            EstadoJuego.estadoJuego.Guardar();


        }
    }
    // Update is called once per frame
    void Update()
    {

    }

    // cambiamos el texto del marcador según los puntos que se
vayan sumando.
    void ActualizarMarcador()
    {
        marcador.text = puntuacion.ToString();
    }
}
```

SCROLL.CS

```
using UnityEngine;

public class Scroll : MonoBehaviour
{
    //esta clase no sirve para iniciar el movimiento del fondo para
dar la sensación de avance



    // variables para iniciar o detener el movimiento y su
velocidad
    public float velocidad = 0f;
```

```
    private bool enMovimiento = false;
    private float tiempoInicio = 0f;

    public bool IniciarEnMovimiento = false;
    // Start is called once before the first execution of Update
after the MonoBehaviour is created
    void Start()
    {
        //no suscribimos a las notificaciones del personaje para
iniciar o detener el movimiento
        NotificationCenter.DefaultCenter().AddObserver(this,
"PersonajeEmpiezaACorrer");
        NotificationCenter.DefaultCenter().AddObserver(this,
"PersonajeAMuerto");

        if (IniciarEnMovimiento)
        {
            PersonajeEmpiezaACorrer();
        }
    }

    // iniciamos el movimiento según la velocidad y justo en el
tiempo que se inicia el movimiento
    void Update()
    {
        if(enMovimiento){

            GetComponent<Renderer>().material.mainTextureOffset
=new Vector2(((Time.time - tiempoInicio) * velocidad) % 1, 0);

        }
    }

    //si el personaje empieza a correr iniciamos el movimiento
    void PersonajeEmpiezaACorrer()
    {
        enMovimiento = true;
        tiempoInicio = Time.time;

    }
    //si el personaje ha muerto lo paramos
    void PersonajeAMuerto()
    {
        enMovimiento = false;
    }
}
```

SEGUIRPERSONAJE.CS

```
using UnityEngine;

public class Seguirpersonaje : MonoBehaviour
{
    // esta clase nos permite fijar la cámara en el personaje y
seguirle según su posición en la pantalla.
    public Transform personaje;
    public float separacion = 6f;



    // vamos cambiando la posición según la posición del personaje
    void Update()
    {
        transform.position = new Vector3 (personaje.position.x +
separacion, transform.position.y, transform.position.z);
    }
}
```

Apunte final, espero que la realización de la tarea este bien realizada, es la primera vez que uso un Mac pero mi ordenador windows se me quedaba corto para ello y ya que lo tenia recién comprado pues lo usé. Espero que no haya ningún problema para ejecutar, yo he probado el juego en Android y funciona bien sin fallos. Lo comentado anteriormente para mi la tarea me ha resultado complicada haya sido por la cuestión que haya sido pues ni yo se jaja. Espero poder aclararme las ideas para el examen. Un saludo.